

Exercício 1 - De caracteres a lexemas e tokens

```
1 import java.util.LinkedHashMap;
2 import java.util.Map;
3
4 public class Main {
5     static final String ALUNO = "Matheu_Previato_Hermenegildo";
6     static final String RA = "24117297-2";
7
8     static final String ENTRADA = "total_mph = valor + 72;";
9
10    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();
11
12    static {
13        SIMBOLOS.put('=', "ATRIBUICAO");
14        SIMBOLOS.put('+', "SOMA");
15        SIMBOLOS.put('-', "SUBTRACAO");
16        SIMBOLOS.put('*', "MULTIPLICACAO");
17        SIMBOLOS.put('/', "DIVISAO");
18        SIMBOLOS.put('.', "PONTO_E_VIRGULA");
19        SIMBOLOS.put('(', "ABRE_PARENTESES");
20        SIMBOLOS.put(')', "FECHA_PARENTESES");
21    }
22
23    public static void main(String[] args) {
24        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
25        System.out.println("ENTRADA: " + ENTRADA);
26        System.out.println();
27        System.out.printf("%-22s %s\n", "LEXEMA", "TOKEN");
28        System.out.println("-----");
29
30        analisar(ENTRADA);
31    }
32 }
```

ALUNO: Matheu_Previato_Hermenegildo | RA: 24117297-2
ENTRADA: total_mph = valor + 72;

LEXEMA	TOKEN
total_mph	IDENTIFICADOR
=	ATRIBUICAO
valor	IDENTIFICADOR
+	SOMA
72	INTEIRO
;	PONTO_E_VIRGULA

Exercício 2 - O que o analisador léxico reconhece

3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33

import java.util.Set;

public class Main {
 static final String ALUNO = "Matheus_Previato_Hermenegildo";
 static final String RA = "24117297-2";

 static final String MPH = "nota_mph";

 static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>(
 Arrays.asList("int", "if", "else", "while", "return")
);

 public static void main(String[] args) {
 System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
 executarTeste("TESTE A - entrada compacta",
 "int total=valor+10;");
 executarTeste("TESTE B - espaços e comentário",
 " int total = valor + 10; // este comentario sera ignorado");
 executarTeste("TESTE C - reservadas e identificadores",
 "int inteiro = 0; while (inteiro < intValor) inteiro = inteiro + 1;");
 executarTeste("TESTE C2 - identificador personalizado",
 "int " + MPH + " = 10;");
 executarTeste("TESTE D - caractere inválido",
 "int valor# = 1;");
 }

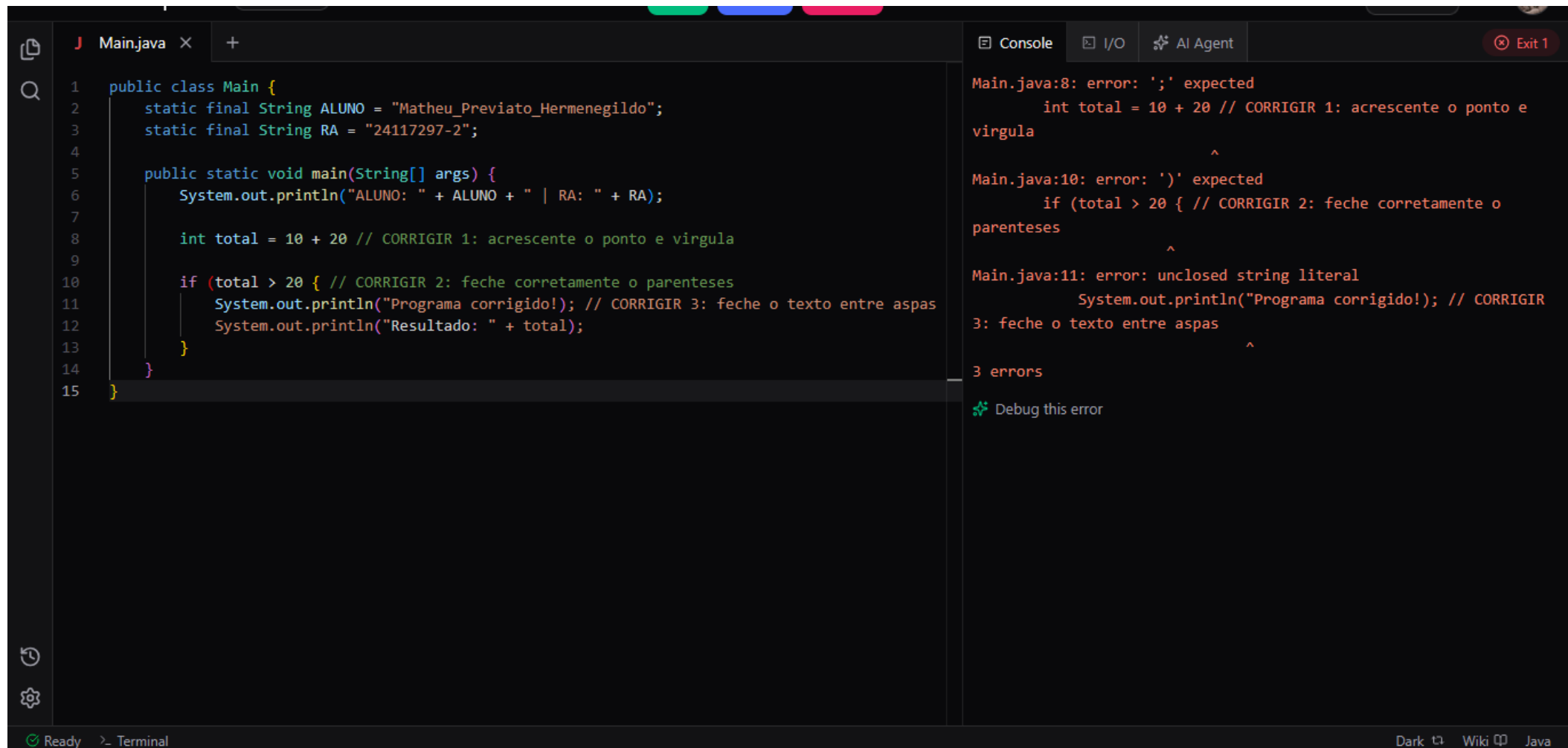
 static void executarTeste(String titulo, String entrada) {
 System.out.println();
 System.out.println("=== " + titulo + " ===");
 System.out.println("ENTRADA: " + entrada);
 System.out.printf("%-22s %-22s %-22s\n", "LEXEMA", "CLASSE", "TOKEN");
 System.out.println("-----");
 String[] tokens = entrada.split(" ");
 for (String token : tokens) {
 String lexema = token;
 String classe = "LETRA";
 String tokenClass = "PALAVRA_RESERVADA";
 if (token.equals("=")) {
 classe = "SIMBOLO";
 tokenClass = "ATRIBUICAO";
 } else if (token.equals("10")) {
 classe = "DIGITO";
 tokenClass = "INTEIRO";
 } else if (token.equals(";")) {
 classe = "SIMBOLO";
 tokenClass = "PONTO_E_VIRGULA";
 } else if (token.equals("int")) {
 classe = "LETRA";
 tokenClass = "PALAVRA_RESERVADA";
 } else if (token.equals("valor")) {
 classe = "LETRA";
 tokenClass = "IDENTIFICADOR";
 } else if (token.equals("#")) {
 classe = "DESCONHECIDO";
 tokenClass = "ERRO_LEXICO";
 }
 System.out.println(lexema + " " + classe + " " + tokenClass);
 }
 }
}

Console
I/O
AI Agent
1.4s

```
inteiro          LETRA          IDENTIFICADOR  
=  
                SIMBOLO         ATRIBUICAO  
inteiro          LETRA          IDENTIFICADOR  
+                SIMBOLO         SOMA  
1                DIGITO          INTEIRO  
;                SIMBOLO         PONTO_E_VIRGULA  
  
=== TESTE C2 - identificador personalizado ===  
ENTRADA: int nota_mph = 10;  
LEXEMA          CLASSE          TOKEN  
-----  
int              LETRA          PALAVRA_RESERVADA  
nota_mph         LETRA          IDENTIFICADOR  
=                SIMBOLO         ATRIBUICAO  
10               DIGITO          INTEIRO  
;                SIMBOLO         PONTO_E_VIRGULA  
  
=== TESTE D - caractere inválido ===  
ENTRADA: int valor# = 1;  
LEXEMA          CLASSE          TOKEN  
-----  
int              LETRA          PALAVRA_RESERVADA  
valor            LETRA          IDENTIFICADOR  
#                DESCONHECIDO    ERRO_LEXICO  
=                SIMBOLO         ATRIBUICAO  
1                DIGITO          INTEIRO  
;                SIMBOLO         PONTO_E_VIRGULA
```

Ready Terminal Dark Wiki Java

Exercício 3 - Quando o compilador encontra erros (erro do compilador)



The screenshot shows an IDE with a Java file named `Main.java`. The code contains several syntax errors, which are highlighted in the console. The code is as follows:

```
1 public class Main {
2     static final String ALUNO = "Matheu_Previato_Hermenegildo";
3     static final String RA = "24117297-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
```

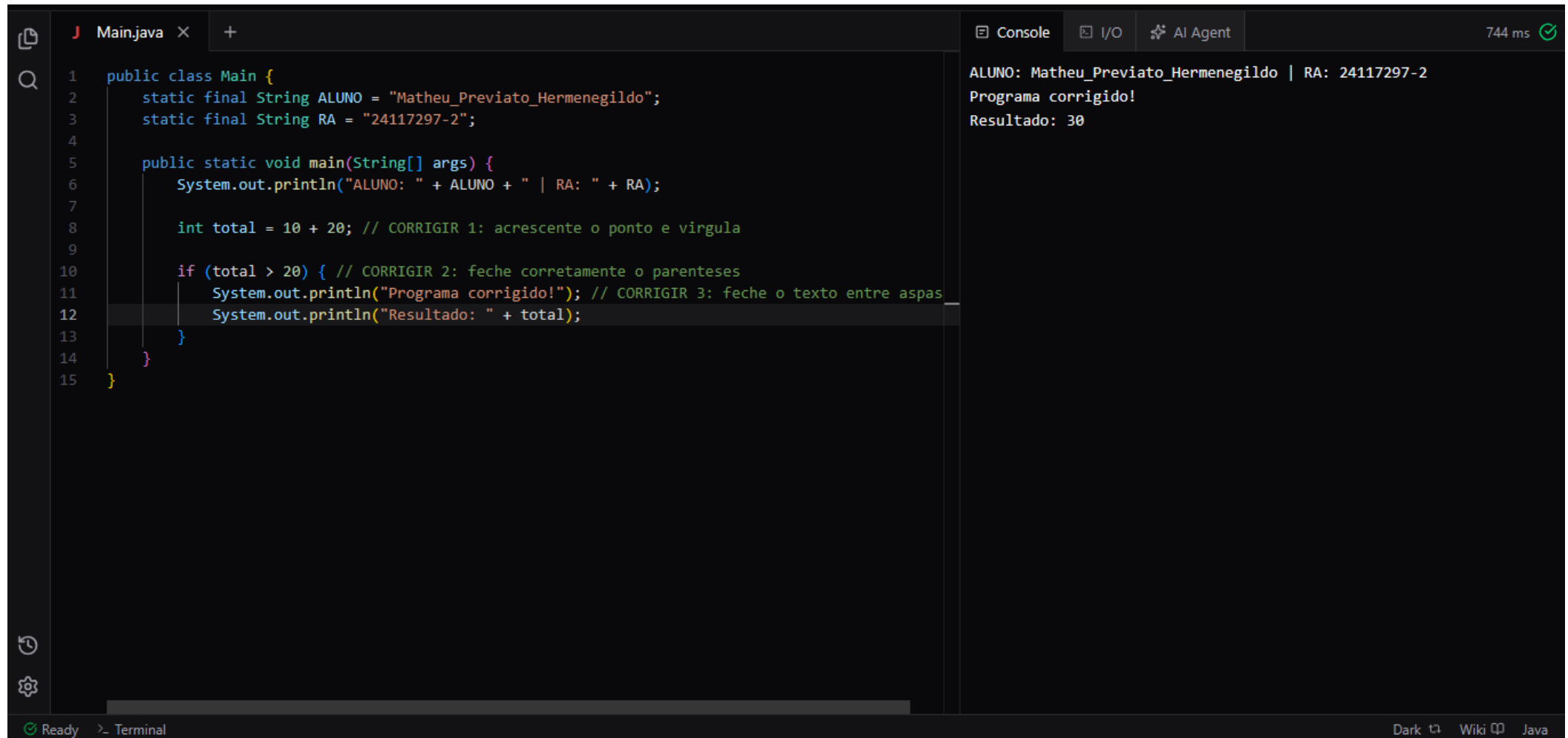
The console shows the following errors:

- `Main.java:8: error: ';' expected`
int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
^
- `Main.java:10: error: ')' expected`
if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
^
- `Main.java:11: error: unclosed string literal`
System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
^

3 errors

Debug this error

Exercício 3 - Quando o compilador encontra erros (programa corrigido)



The screenshot shows an IDE window with a file named `Main.java`. The code is as follows:

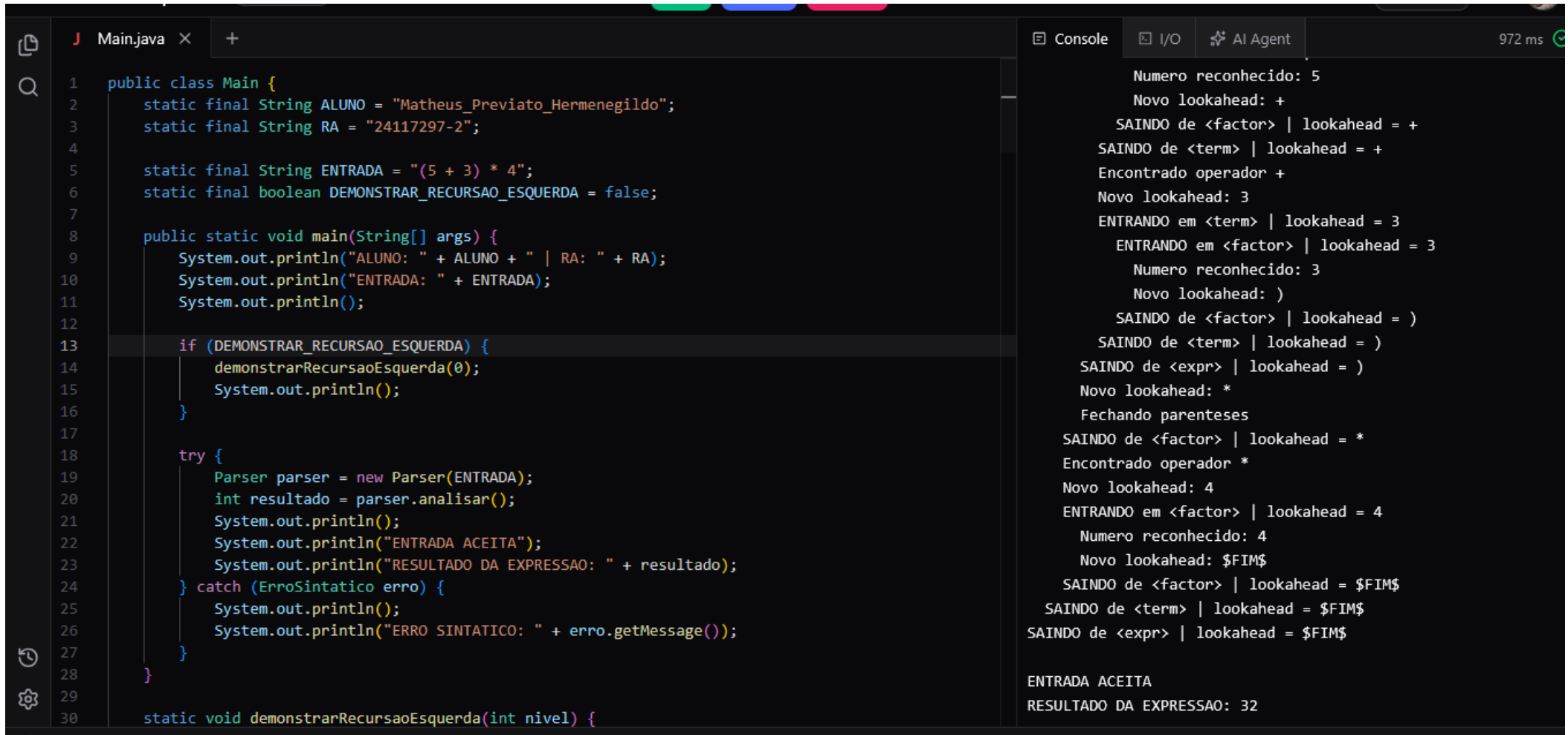
```
1 public class Main {
2     static final String ALUNO = "Matheu_Previato_Hermenegildo";
3     static final String RA = "24117297-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
```

The right-hand pane shows the console output:

```
ALUNO: Matheu_Previato_Hermenegildo | RA: 24117297-2
Programa corrigido!
Resultado: 30
```

The status bar at the bottom indicates the IDE is 'Ready' and the active language is 'Java'.

Exercício 4 - Parser descendente recursivo



The image shows a screenshot of an IDE with a Java file named `Main.java` and a console window. The Java code defines a `Main` class with static variables for student information, input expression, and a flag to demonstrate left recursion. The `main` method prints the input and then checks if left recursion demonstration is enabled. If so, it calls `demonstrarRecursaoEsquerda(0)`. Otherwise, it creates a `Parser` object, analyzes the input, and prints the result. The console output shows the step-by-step execution of the parser, including lookahead updates and the final result of the expression.

```
1 public class Main {
2     static final String ALUNO = "Matheus_Previato_Hermenegildo";
3     static final String RA = "24117297-2";
4
5     static final String ENTRADA = "(5 + 3) * 4";
6     static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;
7
8     public static void main(String[] args) {
9         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10        System.out.println("ENTRADA: " + ENTRADA);
11        System.out.println();
12
13        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
14            demonstrarRecursaoEsquerda(0);
15            System.out.println();
16        }
17
18        try {
19            Parser parser = new Parser(ENTRADA);
20            int resultado = parser.analisar();
21            System.out.println();
22            System.out.println("ENTRADA ACEITA");
23            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
24        } catch (ErroSintatico erro) {
25            System.out.println();
26            System.out.println("ERRO SINTATICO: " + erro.getMessage());
27        }
28    }
29
30    static void demonstrarRecursaoEsquerda(int nivel) {
```

Console Output:

```
Numero reconhecido: 5
Novo lookahead: +
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead: )
SAINDO de <factor> | lookahead = )
SAINDO de <term> | lookahead = )
SAINDO de <expr> | lookahead = )
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: $FIM$
SAINDO de <factor> | lookahead = $FIM$
SAINDO de <term> | lookahead = $FIM$
SAINDO de <expr> | lookahead = $FIM$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 32
```

Exercício 5 - Parser ascendente: deslocar e reduzir

Main.java

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Main {
    static final String ALUNO = "Matheu_Previato_Hermenegildo";
    static final String RA = "24117297-2";

    static final String ENTRADA_VALIDA = "id+id*id";
    static final String ENTRADA_INVALIDA = "id+*id";

    // Na primeira execução, use ENTRADA_VALIDA. Depois troque por ENTRADA_INVALIDA.
    static final String ENTRADA = ENTRADA_VALIDA;

    static final Map<Integer, Map<String, String>> ACTION = new HashMap<>();
    static final Map<Integer, Map<String, Integer>> GOTO = new HashMap<>();
    static final Map<Integer, Regra> REGRAS = new HashMap<>();

    static {
        // Gramática:
        // 1) E -> E + T      2) E -> T
        // 3) T -> T * F      4) T -> F
        // 5) F -> ( E )      6) F -> id
        regra(1, "E", 3, "E -> E + T");
        regra(2, "E", 1, "E -> T");
        regra(3, "T", 3, "T -> T * F");
        regra(4, "T", 1, "T -> F");
        regra(5, "F", 3, "F -> ( E )");
        regra(6, "F", 1, "F -> id");
    }
}
```

Console

I/O

AI Agent

1.4s

PILHA	ENTRADA	ACAO
\$ 0	id+id*id\$	SHIFT id ->
estado 5		
\$ 0 id 5	+id*id\$	REDUCE F ->
id		
\$ 0 F 3	+id*id\$	REDUCE T -> F
\$ 0 T 2	+id*id\$	REDUCE E -> T
\$ 0 E 1	+id*id\$	SHIFT + ->
estado 6		
\$ 0 E 1 + 6	id*id\$	SHIFT id ->
estado 5		
\$ 0 E 1 + 6 id 5	*id\$	REDUCE F ->
id		
\$ 0 E 1 + 6 F 3	*id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*id\$	SHIFT * ->
estado 7		
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT id ->
estado 5		
\$ 0 E 1 + 6 T 9 * 7 id 5	\$	REDUCE F ->
id		
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T
* F		
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E
+ T		
\$ 0 E 1	\$	ACCEPT

Ready

Terminal

Dark

Wiki

Java